ZK from Zero on Cardano

2025

by ElRaulito

I FOUNDATIONS OF ZERO-KNOWLEDGE 2

1 INTRODUCTION TO ZERO-KNOWLEDGE PROOFS 3

1.1 What are Zero-Knowledge Proofs? 3

1.2 Core Concepts 4

1.3 Types of Zero-Knowledge Proof Systems 5

1.4 ZK on Cardano 5

II ORIGINS AND EVOLUTION 6

2 THE HISTORY OF ZK 7

2.1 The Origins of Zero-Knowledge 7

2.2 From Theory to Practice 7

2.3 The SNARK Revolution 8

2.4 Modern ZK: STARKs and Beyond 8

2.5 ZK Arrives on Cardano 9

III ZK IN WEB3: PRIVACY COINS AND BEYOND 11

3 ZK IN THE WILD 12

3.1 Why Web3 Needs ZK 12

3.2 Zcash 12

3.3 Monero 14

3.4 Other ZK Applications in Web3 15

3.5 Midnight 16

IV ZK FOR COMPRESSION AND PRIVACY 21

4 PERFORMANCE, PRIVACY AND TRADE-OFFS 22

4.1 ZK for Data Minimisation and Proof Compression 22

4.2 Succinctness and Scalability 23

4.3 Privacy by Design: Applications and Limitations 24

FOUNDATIONS OF ZERO-KNOWLEDGE

1. INTRODUCTION TO ZERO-KNOWLEDGE PROOFS

1.1. WHAT ARE ZERO-KNOWLEDGE PROOFS?

All of us have heard about Zero-Knowledge, but what is it exactly? In my experience, many projects have abused this term — using it only to hide something already known, like multi-signature protocols, or centralising data and value behind a web2 backend while claiming ZK legitimacy.

A telling example: a project once claimed to make all transactions anonymous. You would send stablecoins to them, tell them the intended receiver, and they would forward the funds bundled with other transfers so the origin was obscured. There is no ZK in that process — it is simply a centralised mixer.

Where's Waldo and the Hash Lock Analogy

The example that first made ZK click for me is **Where's Waldo** (known as *Where's Wally* outside North America). Imagine you have found Waldo on a busy page but you want to prove to a friend that you know where he is without pointing at him. You could take a very large sheet of paper, cut a small hole over Waldo, and show just his face through the hole. Your friend sees Waldo and is convinced you know the location — without learning anything else about the page.

Using secrets in blockchains was already possible in Bitcoin with **hash-lock scripts**: you choose a password known only to you, hash it, and lock funds to that hash. Anyone who can reveal the pre-image unlocks the coins.

> Hash Time-Locked Contracts (HTLCs) are the technology behind cross-chain atomic swaps — used by protocols such as *Fluid Tokens* — and underpins payment routing on the Bitcoin Lightning Network.

The critical limitation of hash locks is that revealing the password burns it: once the pre-image is public, that address is permanently unsafe because anyone now knows the secret tied to it. With ZK you can **prove that you know the password without ever revealing it**. That is the real magic.

ZK as Compression, Not Just Privacy

Most people stop at the privacy angle. ZK is far more useful as a **compression tool**, and this is arguably its most important application in blockchain today.

ZK ROLLUPS (GENERAL CASE).

An L2 network may process thousands of transactions per second. Posting every transaction to L1 would be prohibitively expensive. Instead, the rollup posts only a ZK proof attesting that all those transactions were correctly signed by the users involved. An L1 smart

contract verifies the proof — a single compact check replaces thousands of individual verifications.

> **Thousands of L2 transactions ⟶ one ZK proof on L1.**
> The on-chain contract only verifies the proof. Nobody can lie about what happened.

THE CARDANO USE CASE.

When you interact with a Cardano DEX today, the contract executes all calculations on-chain: it checks amounts, validates prices, and enforces invariants for *every* order, consuming script budget. With ZK the flow changes completely:

1. You build the transaction and perform all calculations *off-chain* — the amounts to receive, slippage bounds, fee splits.
2. You attach to your transaction a ZK proof of those computations.
3. The DEX contract checks only that the proof is valid. It does not re-execute the math.

Because verification is orders of magnitude cheaper than re-computation, a single transaction can batch hundreds of orders. You see? ZK is being used to **compress computation**, not to hide anything.

1.2. RE CONCEPTS

Before going further it is worth fixing the three properties that every honest ZK system must satisfy.

- **Completeness.** If the statement is true and the prover is honest, the verifier will always be convinced.
- **Soundness.** If the statement is false, no cheating prover can convince the verifier (except with negligible probability).
- **Zero-Knowledge.** The verifier learns nothing beyond the truth of the statement. The proof reveals no information about the secret witness.

In every ZK protocol there are two parties:

- **Prover** — knows a secret (the *witness*) and wants to convince the verifier without revealing it.
- **Verifier** — checks the proof and decides whether to accept it.

teractive vs. Non-Interactive Proofs

Early ZK proofs were *interactive*: the verifier sends random challenges and the prover responds, repeating rounds until the verifier is statistically convinced. This is elegant in theory but impractical on a blockchain where there is no live back-and-forth.

The **Fiat–Shamir heuristic** (1986) transforms interactive protocols into *non-interactive* ones: the prover generates the verifier's challenges themselves by hashing the transcript so far. The resulting proof can be verified by anyone, at any time, without interaction — exactly what on-chain verification requires.

1.3. TYPES OF ZERO-KNOWLEDGE PROOF SYSTEMS

The ZK landscape has exploded in the past decade. Here is a practical map of the main families.

- **zk-SNARKs** (Succinct Non-interactive Arguments of Knowledge) — very small proofs and fast verification, but require a trusted setup ceremony. Groth16 and PLONK are the dominant constructions. Used in Zcash, Ethereum's zkSync and Scroll.
- **zk-STARKs** (Scalable Transparent Arguments of Knowledge) — no trusted setup, post-quantum secure, but larger proof sizes. Used by StarkWare and StarkNet.
- **Bulletproofs** — no trusted setup, compact range proofs, but verification is slower and scales linearly with circuit size. Used in Monero and Mimblewimble.
- **PLONK and universal SNARKs** — use a single trusted setup that works for *any* circuit up to a given size, removing the need for per-circuit ceremonies.

System	Trusted Setup	Proof Size	Verification
Groth16	Per-circuit	∼200 bytes	Very fast
PLONK	Universal	∼500 bytes	Fast
STARKs	None	∼45–200 KB	Moderate
Bulletproofs	None	∼1 KB	Slow (linear)

1.4. ZK ON CARDANO

Cardano's **eUTxO model** is well-suited to ZK verification: a script validator receives the proof as part of the redeemer or datum and checks it in a single deterministic execution. Because Cardano contracts run off-chain before submission, the prover can generate the proof locally and attach it — the on-chain script only verifies.

With **Plutus V3**, Cardano gained native cryptographic primitives for the BLS12-381 elliptic curve pairing:

- `bls12_381_G1_add`, `bls12_381_G2_add` — elliptic curve point addition in $\mathbb{G}_1$ and $\mathbb{G}_2$.
- `bls12_381_millerLoop`, `bls12_381_finalVerify` — pairing operations needed to verify Groth16 proofs on-chain.

These primitives are also exposed in **Aiken**'s standard library under `aiken/crypto/bls12_381`, making it practical to write on-chain Groth16 verifiers in a high-level language.

The goal of this book is to start from the very basics — proving to someone that you know a value and can therefore spend a UTxO — and build up to more complex applications: compressing off-chain computation, and eventually constructions similar to privacy protocols (studied here for academic purposes only).

ORIGINS AND EVOLUTION

2. THE HISTORY OF ZK

2.1. THE ORIGINS OF ZERO-KNOWLEDGE

Zero-Knowledge proofs were invented in **1985** by Shafi Goldwasser, Silvio Micali, and Charles Rackoff at MIT. Their paper — *The Knowledge Complexity of Interactive Proof Systems* — introduced the concept of an interactive proof where a prover convinces a verifier of the truth of a statement while revealing nothing beyond that truth. The paper won the Gödel Prize in 1993 and is widely considered one of the most influential works in the history of cryptography.

At the time, the result was purely theoretical. The proof systems described required many rounds of interaction, were expensive to compute, and existed only to answer a complexity-theoretic question: *how much "knowledge" does a proof leak?* The answer, in some cases, was: none.

2.2. FROM THEORY TO PRACTICE

The Schnorr Protocol

A landmark step toward practicality came with Claus-Peter Schnorr's identification scheme (1989). The **Schnorr protocol** is a three-move sigma protocol:

1. The prover commits to a random value.
2. The verifier sends a random challenge.
3. The prover responds with a value that combines the challenge with their secret.

The verifier can check the response without learning the secret. Sigma protocols like this are the ancestors of almost every practical ZK system in use today.

The Fiat-Shamir Heuristic

In 1986, Amos Fiat and Adi Shamir showed how to convert any sigma protocol into a **non-interactive** proof by replacing the verifier's random challenge with a hash of the prover's commitment. The resulting proof can be generated offline and verified by anyone — no live interaction needed. This single insight is what makes ZK proofs viable on blockchains.

Schnorr Signatures on Cardano with FROST

Schnorr signatures eventually became a production cryptographic primitive. Today you can use them on Cardano through the **FROST** threshold signature protocol, accessible at `https://www.frost.express/`. FROST allows a group of participants to collaboratively sign a message with a Schnorr signature in a way that is indistinguishable from a single-signer scheme on-chain, with no single party holding the full private key.

2.3. THE SNARK REVOLUTION

Through the 2000s and early 2010s, researchers worked to make ZK proofs efficient enough for real applications. The breakthrough came in 2013–2016 with a family of constructions called **zk-SNARKs**.

TRUSTED SETUP AND GROTH16.

The most efficient SNARK construction is **Groth16** (Jens Groth, 2016). It produces proofs of only ∼200 bytes that verify in milliseconds — fast enough for on-chain use. The catch is a *trusted setup ceremony*: a group of participants must generate public parameters, and if any participant keeps their secret "toxic waste", they could forge proofs. In practice, multi-party ceremonies with hundreds of participants make this risk negligible.

ZCASH AND REAL-WORLD DEPLOYMENT.

Zcash (2016) was the first large-scale deployment of zk-SNARKs. It uses Groth16 to let users make shielded transactions: the blockchain verifies a proof that the sender has enough funds and the transaction is balanced, without revealing amounts or addresses.

UNIVERSAL SETUPS.

Circuit-specific trusted setups are impractical for general use. **PLONK** (2019) and **Marlin** introduced *universal* setups: a single ceremony valid for any circuit up to a given size. This enabled the modern ecosystem of general-purpose zkVMs and zkEVMs.

2.4. MODERN ZK: STARKS AND BEYOND

STARKS.

Eli Ben-Sasson and colleagues at the Technion introduced **zk-STARKs** in 2018. STARKs require *no trusted setup* — all parameters are public and derived from collision-resistant hash functions. They are also believed to be post-quantum secure. The trade-off is proof size: a STARK proof can be tens to hundreds of kilobytes versus a few hundred bytes for a SNARK.

StarkWare built an entire ecosystem around STARKs: *StarkEx* powers major DeFi protocols, and *StarkNet* is a permissionless L2 on Ethereum.

ZK ROLLUPS.

The rollup model took off around 2020–2022. **zkSync**, **Scroll**, and **Polygon zkEVM**

all use ZK proofs to compress thousands of Ethereum transactions into a single on-chain verification, achieving Ethereum security with a fraction of the gas cost.

RECENT DEVELOPMENTS.

The frontier is moving fast: *folding schemes* like **Nova** (2021) allow recursive proof composition with near-zero overhead; *lookup arguments* like Plookup dramatically reduce the cost of proving table lookups and bit operations; and *zkVMs* such as RISC Zero let developers write programs in Rust and get a ZK proof of correct execution essentially for free.

2.5. ARRIVES ON CARDANO

utus V3 and BLS12-381 Primitives

With the Chang hard fork (2024), Cardano gained **Plutus V3** and a set of native BLS12-381 elliptic curve primitives. These allow on-chain verification of Groth16 proofs without any workarounds. The primitives are available in Aiken's standard library:

- `aiken/crypto/bls12_381/g1` — operations on the $\mathbb{G}_1$ group
- `aiken/crypto/bls12_381/g2` — operations on the $\mathbb{G}_2$ group
- Pairing operations: `miller_loop` and `final_verify`

See the full API at `https://aiken-lang.github.io/stdlib/aiken/crypto/bls12_381/g1.html`.

nus Wallet

The most complete open-source ZK project on Cardano today is **Janus Wallet**, available at:

`https://github.com/leobel/janus-wallet`

Janus demonstrates a working Groth16 verifier on Cardano: a user generates a ZK proof off-chain that they know a secret, and the on-chain Aiken validator verifies the proof and releases the UTxO. It is the closest thing Cardano has to a production-grade ZK smart contract today.

at This Book Covers

The rest of this book is structured as a progressive journey:

1. **Starting from basics** — proving knowledge of a secret value and using a ZK proof to spend a UTxO on Cardano.
2. **Compression** — using off-chain computation with on-chain verification to batch many operations into a single transaction.

3. **Advanced constructions** — privacy-preserving patterns studied for academic purposes. All material here is for educational use. This book does not promote protocols designed to obscure transaction data.

ZK IN WEB3: PRIVACY COINS AND BEYOND

3. ZK IN THE WILD

3.1. WHY WEB3 NEEDS ZK

Blockchain is beautiful precisely because it is transparent — but that same transparency can be turned against you.

In 2022, Tornado Cash was placed on the US government's sanctions list. If your address had interacted with it in any transaction, exchanges could freeze your assets. Worse, someone could *send* you a transaction through Tornado Cash and get your address blacklisted without your consent. Once your address is linked to your real identity, every trade, transfer, and incoming payment becomes traceable. Dirty money from hacks could be sent to you on purpose.

This is why on-chain privacy matters — not to hide wrongdoing, but to protect ordinary users from targeted attacks, surveillance, and financial coercion.

Bitcoin Privacy Practices

Bitcoin has no built-in privacy layer, but the community has developed a set of good practices over the years. I believe these can be replicated straightforwardly on Cardano and are worth stating clearly:

- **Never reuse an address.** Bitcoin and Cardano both support deriving unlimited addresses from a single seed. Each payment should receive a fresh address.
- **Avoid round-number amounts and self-change.** Sending exactly 1 BTC and returning change to yourself are fingerprints that chain analytics tools look for.
- **Use multiple inputs and outputs.** A transaction that mixes several UTxOs from different addresses and fans out to several recipients is harder to trace than a one-to-one transfer.
- **Isolate KYC funds.** If you received ADA from a KYC exchange, keep that UTxO entirely separate from addresses you want to keep private. Mixing them collapses your privacy instantly.

These practices reduce linkability but cannot eliminate it. For stronger guarantees you need a cryptographic construction — which is where ZK enters the picture.

3.2. ZCASH

Zcash gained mainstream attention during the Silk Road era: because Silk Road was a black market where participants needed payment privacy, a coin with strong cryptographic guarantees was attractive. Unfortunately the listing of Zcash was front-run by the US government's take-down and arrest of Ross Ulbricht, the founder of Silk Road, before Zcash could see widespread use there. Ross spent ten years in prison; the last US president promised to pardon him during the campaign and ultimately did — he is now a free man.

> The story of Ross Ulbricht is a reminder that financial privacy and personal freedom are deeply connected. Regardless of your view of Silk Road, the case set important legal precedents about online speech, sentencing, and the use of blockchain evidence.

How Zcash Works

Zcash uses **zk-SNARKs** (specifically the Groth16 construction) to enable shielded transactions. Every transaction on Zcash involves two possible address types:

- **Transparent addresses** (t-addresses) — behave like Bitcoin addresses; all amounts and addresses are public.
- **Shielded addresses** (z-addresses) — balances and transaction details are hidden behind a ZK proof.

When you send ZEC from one shielded address to another, the transaction contains a proof that:

1. You own a note (a UTXO) with sufficient value.
2. The total value in equals the total value out (no inflation).
3. You are not double-spending.

The verifier (the entire network) checks only the proof. Amounts, sender, and receiver remain invisible. The construction relies on a **trusted setup ceremony** — a multi-party computation that generates the public parameters. If every participant in the ceremony destroys their toxic waste, the parameters are safe. Zcash ran the Sapling ceremony in 2018 with hundreds of participants.

> **Zcash in one sentence:** a proof that your transaction is valid, without revealing who you are, how much you sent, or who received it.

The main criticism of Zcash historically has been low adoption of shielded addresses. Most ZEC on exchanges sits in transparent addresses, which undermines the anonymity set. Shielded usage has grown since the Sapling upgrade, but full privacy still requires both sender and receiver to use z-addresses.

3.3. MONERO

Monero takes a different philosophical stance from Zcash: **privacy by default for every transaction**, not opt-in. There are no transparent addresses. Every Monero transaction is private.

Monero achieves this through three complementary mechanisms:

- **Ring Signatures.** When you spend XMR, your transaction is signed by a ring of n possible signers drawn from the UTXO set. An observer cannot tell which ring member is the true sender.
- **Stealth Addresses.** Each transaction generates a one-time destination address derived from the recipient's public key. Only the recipient can link it to their wallet.
- **Confidential Transactions (RingCT / Bulletproofs).** Transaction amounts are hidden using Pedersen commitments. A Bulletproof proves that all amounts are positive and that inputs equal outputs, without revealing the values. Bulletproofs require no trusted setup.

Feature	Zcash (ZEC)	Monero (XMR)
Launch Year	2016	2014
Founders	Zooko Wilcox-O'Hearn & Electric Coin Company	Nicolas van Saberhagen (pseudonymous)
Privacy Mechanism	zk-SNARKs (zero-knowledge proofs)	Ring Signatures, Stealth Addresses, RingCT
Privacy Type	Optional (transparent or shielded)	Mandatory (always private)
Consensus Mechanism	Proof-of-Work (Equihash algorithm)	Proof-of-Work (RandomX algorithm)
Total Supply	21 million ZEC	~18.4 million XMR with tail emission
Development Organizations	Electric Coin Company & Zcash Foundation	Monero Core Team & Community Developers
Smart Contract Support	Limited (possible via cross-chain integrations)	None (focused purely on privacy transactions)
Regulatory Accessibility	Higher due to optional transparency	Lower due to enforced anonymity
2025 Market Capitalization	$10.8 billion USD	$7.8 billion USD

Datawallet

Zcash vs. Monero at a glance:

	Zcash	Monero
Privacy model	Opt-in (shielded)	Mandatory (always on)
Proof system	zk-SNARK (Groth16)	Bulletproofs (no ZK circuit)
Trusted setup	Yes (Sapling)	No
Anonymity set	All shielded UTxOs	Ring of n UTxOs
Proof size	~200 bytes	~2 KB

Monero's mandatory privacy means every user contributes to the anonymity set, which is a fundamental advantage. The trade-off is that ring signatures scale linearly, and proving range correctness with Bulletproofs is slower than a SNARK verifier.

3.4. OTHER ZK APPLICATIONS IN WEB3

ZK proofs have spread far beyond privacy coins. Here are the most important application domains and projects.

Mina Protocol

Mina describes itself as the world's lightest blockchain. Instead of storing the full chain history, every node stores only a **constant-size ZK proof** (currently 22 KB) that the entire chain is valid. This proof is recursive: when a new block is produced, the block producer generates a proof that the new state is valid *and* that the previous proof was valid. The chain compresses down to a single proof of everything that ever happened.

This is a stunning result: syncing a Mina node takes seconds regardless of chain age, because you only verify one proof.

Aztec

Aztec is a privacy-first ZK rollup on Ethereum. Its **Noir** language lets developers write circuits in a high-level, Rust-like syntax. Aztec's architecture separates:

- **Private functions** — executed entirely client-side, producing ZK proofs, with no on-chain trace.
- **Public functions** — executed by the sequencer, visible on-chain.

Applications on Aztec can mix both: a private transfer function hides the sender and amount, while a public function updates a visible registry. Noir compiles to ACIR (Abstract Circuit Intermediate Representation), which multiple backends can prove.

Polygon zkEVM

Polygon zkEVM is a ZK rollup that is **EVM-equivalent**: Ethereum smart contracts and tooling work without modification. Every batch of transactions is accompanied by a zk-SNARK proof that the EVM state transition was correct. The L1 Ethereum contract verifies the proof in a single transaction, achieving Ethereum-level security with dramatically lower gas costs per operation.

StarkNet

StarkNet uses **zk-STARKs** (no trusted setup, post-quantum secure) and its own language, Cairo, which compiles to a provable virtual machine. STARKs produce larger proofs than SNARKs but scale better for very large computations and avoid the trusted-setup risk entirely.

3.5. MIDNIGHT

Midnight is a **data-protection blockchain** developed by Input Output Global (IOG) as a partner chain to Cardano. Its goal is to let developers build applications that can *selectively disclose* information while keeping everything else private, using zero-knowledge proofs as the underlying engine.

What Midnight Enables

- **Private Trading.** Decentralized exchanges where order size and execution timing remain hidden from the public ledger.
- **Confidential DeFi.** Private lending, portfolio management, and liquidity pools where balances are shielded.
- **Compliant Identity (ZK-KYC).** Prove that you are over 18, or that you are an accredited investor, without revealing your passport or income documents to every counterparty.
- **Shielded and Unshielded Tokens.** Developers can create tokens that keep balances private by default, or public when the application requires it.
- **Compact Smart Contracts.** Midnight uses a TypeScript-based language for writing privacy-preserving applications, keeping the developer experience accessible.

How Midnight Infrastructure Works

Running on Midnight requires two services working together:

- **Node** — connects to the Midnight network, participates in consensus, and submits transactions.
- **Indexer** — reads data from the node and exposes it through a **GraphQL API**.

> The biggest architectural difference from Cardano's REST-based APIs is that the Midnight indexer speaks **GraphQL**. Instead of hitting specific endpoints, you write a query describing exactly what you want, and the API returns only that data. This means fewer round trips and more flexibility, but requires a different mental model if you are used to REST.

Blockfrost hosts both the node and indexer as a managed service across three environments:

Environment	Use
Mainnet	Production
Preview	Testing
Preprod	Testing

The three service endpoints for Mainnet are:

Service	Endpoint
Indexer API	`https://midnight-mainnet.blockfrost.io/api/v0/`
WebSocket	`wss://midnight-mainnet.blockfrost.io/api/v0/ws`
Node RPC	`https://rpc.midnight-mainnet.blockfrost.io`

The Indexer API handles queries for blocks, transactions, contracts, and wallet events. The WebSocket supports real-time subscriptions for block streaming, contract actions, and ledger events. The Node RPC provides direct JSON-RPC access for wallet providers and transaction submission.

Authentication

There are two ways to authenticate with Blockfrost's Midnight endpoints:

- **Header:** pass your API key as `project_id` in the request header (same as Cardano).
- **Query parameter:** append `?project_id=YOUR_PROJECT_ID` to the URL — useful for clients that cannot set custom headers, including Midnight.js itself.

When configuring Midnight.js to use Blockfrost, pass the full URL with the project ID embedded:

```
https://midnight-mainnet.blockfrost.io/api/v0/?project_id=YOUR_PROJECT_ID
```

Getting Started with Midnight.js

Midnight.js is the official SDK for building on Midnight. It handles contract deployment, private state management, ZK proof generation, and transaction submission.

PREREQUISITES.

- Node.js 22.x or higher
- Docker (required to run the local proof server)
- A Blockfrost account with a Midnight project (free tier available)

INSTALLATION.

Install the full SDK barrel package:

```
npm install @midnight-ntwrk/midnight-js
```

Or install individual packages if you only need specific capabilities:

```
npm install @midnight-ntwrk/midnight-js-types
npm install @midnight-ntwrk/midnight-js-contracts
npm install @midnight-ntwrk/midnight-js-network-id
```

STEP 1 – SET THE NETWORK.

Before any operation, configure which network you are targeting:

```
import { setNetworkId } from '@midnight-ntwrk/midnight-js-network-id';

setNetworkId('preprod'); // or 'mainnet', 'preview'
```

STEP 2 – CONFIGURE PROVIDERS.

Midnight.js uses a modular provider model. Each provider handles one concern:

```
import { levelPrivateStateProvider }
  from '@midnight-ntwrk/midnight-js-level-private-state-provider';
import { indexerPublicDataProvider }
  from '@midnight-ntwrk/midnight-js-indexer-public-data-provider';
import { FetchZkConfigProvider }
  from '@midnight-ntwrk/midnight-js-fetch-zk-config-provider';
import { httpClientProofProvider }
  from '@midnight-ntwrk/midnight-js-http-client-proof-provider';

const blockfrostUrl =
  'https://midnight-preprod.blockfrost.io/api/v0'
  + '?project_id=YOUR_PROJECT_ID';

const providers = {
  // Encrypted local storage for your private state
  privateStateProvider: levelPrivateStateProvider({
    storePath: './my-private-state'
  }),
  // GraphQL queries to the Midnight indexer
  publicDataProvider: indexerPublicDataProvider(blockfrostUrl),
  // Fetches ZK artifacts (prover keys, verifier keys)
  zkConfigProvider: new FetchZkConfigProvider('/zk-artifacts'),
  // Sends circuits to the proof server for proving
  proofProvider: httpClientProofProvider('http://localhost:6300')
};
```

The proof server runs locally in Docker. It receives your circuit inputs and returns a ZK proof that can be attached to a transaction. Your private inputs never leave your machine.

STEP 3 – DEPLOY A CONTRACT.

```
import { deployContract }
  from '@midnight-ntwrk/midnight-js-contracts';
import { compiledContract } from './your-contract';

const deployed = await deployContract(providers, {
  compiledContract,
  privateStateId: 'my-state',
  initialPrivateState: { counter: 0n }
});

console.log('Contract deployed at:', deployed.deployTxData.contractAddress);
```

STEP 4 – CALL A CIRCUIT (PRIVATE TRANSACT

```
import { findDeployedContract }
  from '@midnight-ntwrk/midnight-js-contracts';

const contract = await findDeployedContract(providers, {
  contractAddress: deployed.deployTxData.contractAddress,
  compiledContract,
  privateStateId: 'my-state'
});

// Calling a circuit generates a ZK proof locally,
// then submits the transaction to the network.
const tx = await contract.callTx.increment();
await tx.wait();
\end{verbatim>}

\paragraph{Step 5 --- Query state.}

\begin{verbatim}
import { getPublicStates }
  from '@midnight-ntwrk/midnight-js-contracts';

const states = await getPublicStates(providers, contractAddress);
console.log('Current public state:', states);
```

Key Concepts to Understand

- **Circuits** — the functions you write in Compact (Midnight's language) that run locally and generate ZK proofs. Think of them as the private business logic.
- **Private state** — data stored on your device, encrypted with AES-256-GCM via LevelDB. It never appears on the blockchain.
- **Ledger state** — the public on-chain state of a contract. Anyone can read it; nobody can fake it.
- **ZK artifacts** — files produced by the Compact compiler: prover keys, verifier keys, and the ZKIR (Zero-Knowledge Intermediate Representation). These are loaded at runtime.

- **Transaction flow** — local execution of the circuit → proof generation by the proof server → balancing dust token fees → broadcast to the network.

Creating a Blockfrost Account

If you already have a Blockfrost account, Midnight support is included at no extra cost — just create a new project and select Midnight Mainnet. If you are new:

1. Sign up at `https://blockfrost.io` (free tier: 50,000 requests per day, no credit card required).
2. Click **Add Project** and select **Midnight Mainnet**.
3. Copy your API key and use it in the URL as shown above.

> Midnight requests share your existing Blockfrost rate limit. The GraphQL API also enforces per-query limits on depth, number of fields, complexity, and execution timeout. Requests that exceed these limits return an error describing which limit was hit.

For full SDK documentation see `docs.midnight.network/sdks/official/midnight-js` and for the Blockfrost-specific configuration see `docs.blockfrost.io/midnight/`.

ZK FOR COMPRESSION AND PRIVACY

4. PERFORMANCE, PRIVACY AND TRADE-OFFS

4.1. FOR DATA MINIMISATION AND PROOF COMPRESSION

A point that is not stressed nearly enough: **ZK is not only about privacy**. In physics, light behaves as both a particle and a wave depending on how you observe it. ZK has a similarly dual nature — it can hide information, but it can also *compress* it. These two properties are completely independent, and in practice the compression use case is arguably more economically important today.

The Core Idea

Suppose an L2 network processes 100,000 transactions in one hour. Each transaction has a sender, a receiver, an amount, and a signature. If you posted all of that data to L1, you would overwhelm it and pay enormous fees.

The ZK alternative: the L2 executes all 100,000 transactions off-chain, then generates a single **ZK proof** attesting that every transaction was correctly signed by the legitimate owner of each wallet. The L1 smart contract verifies only the proof — a task that costs a fixed amount of gas regardless of how many transactions are bundled inside.

> **100,000 L2 transactions $\longrightarrow$ one ZK proof on L1.**
> The proof is a few kilobytes. The data it certifies could be gigabytes.

This is called a **ZK Rollup**. Nobody can lie about what happened: if the proof verifies, the state transition is correct. The blockchain does not need to see the individual transactions at all.

Why This Matters for Cardano

Consider a Cardano DEX. Today, every order executes on-chain: the validator checks amounts, validates prices, and enforces invariants for each UTxO — consuming script budget per order. With ZK the flow becomes:

1. The DEX batches many orders off-chain.
2. It generates a ZK proof that all orders were correctly matched and all balances balance.
3. A single on-chain transaction carries only the proof and the net state changes.

The on-chain validator verifies the proof. It does not re-execute the matching logic. This is orders of magnitude cheaper per order and allows throughput that would be impossible otherwise.

The Cost: Proof Generation Time

ZK proofs are not free to generate. The off-chain prover must run the circuit — which is an arithmetic encoding of your computation — over the full witness data. For large

circuits this can take seconds to minutes on consumer hardware.

This is the fundamental trade-off of ZK compression: *the prover pays a large upfront computational cost so that the verifier pays almost nothing.* In a rollup setting the prover is a well-resourced sequencer, so the cost is amortised across all the transactions it bundles. For a single-user proof the user's device bears the cost.

4.2. SUCCINCTNESS AND SCALABILITY

The "S" in zk-SNARK stands for **Succinct**. A succinct proof has two properties:

- **Small size.** A Groth16 proof is ∼200 bytes regardless of the size of the computation it certifies. A PLONK proof is a few hundred bytes. A STARK proof is larger (∼50–200 KB) but still bounded.
- **Fast verification.** Verifying a proof takes milliseconds and a fixed amount of compute — again, regardless of the underlying computation size.

This asymmetry is what makes ZK interesting for blockchains. The blockchain is a slow, expensive, globally-replicated computer. You want to put as little work on it as possible. Succinctness lets you offload arbitrarily large computations and pay only a constant verification cost on-chain.

Recursive Proofs and Proof Aggregation

The compression story gets even more powerful with **recursive proofs**: a proof that another proof is valid. If you can verify a ZK proof inside a ZK circuit, you can build a tree of proofs:

- Each leaf proves a batch of transactions.
- Each internal node proves that its two children proofs are valid.
- The root is a single proof attesting to the entire tree.

This is how Mina compresses its entire blockchain history into 22 KB, and how recursive SNARKs like Nova allow continuous proof accumulation with near-zero overhead per step. The on-chain verifier always sees only one proof, regardless of how deep the tree is.

Scalability in Practice

Approach	Throughput	Security
Pure L1	Limited by block size	Native L1 security
Optimistic Rollup	10×–100×	L1 security + fraud proof delay
ZK Rollup	100×–1000×	L1 security + immediate finality
Recursive ZK	Near unlimited batching	Same as ZK Rollup

ZK Rollups have an important advantage over Optimistic Rollups: there is **no challenge window**. An Optimistic Rollup has to wait days before a withdrawal is final, to give time

for fraud proofs. A ZK Rollup is final as soon as the proof verifies — typically within minutes of the batch being posted.

4.3. PRIVACY BY DESIGN: APPLICATIONS AND LIMITATIONS

Building with privacy is genuinely hard — not mathematically, but in terms of user experience.

When you build a standard DApp, you can see everything about your user: their balance, their history, their open positions. You can use this data to personalise the interface, warn them about risks, guide them through deposit flows, and show them relevant information. The transparency of the blockchain is, paradoxically, a *UX feature* for most applications.

The moment you introduce privacy, that transparency disappears. You can no longer read the user's balance. You cannot verify their eligibility client-side without the user handing you a proof. The interface has to work without the information it used to take for granted.

Tornado Cash as an Academic Case Study

> This book does not promote or endorse the use of Tornado Cash or any sanctioned platform. The following is a purely technical and UX analysis for educational purposes.

Tornado Cash is the clearest example of where ZK privacy meets UX friction. The flow for a user was:

1. Deposit funds into the contract. Receive a **note** — a file containing a cryptographic secret that proves ownership of the deposit.
2. Store the note securely (it is the only way to redeem; lose it and the funds are gone).
3. Later, from a different address with no connection to the original, present the note and a ZK proof that you know the secret. The contract releases the funds to a fresh address.

The cryptography is elegant. The UX is a disaster for non-technical users. Downloading a file, storing it safely, transferring it across devices, and then running a separate withdrawal flow is completely alien to anyone who has not used a command-line wallet.

The cypherpunk community loves this kind of design — you are in full control, you hold your own keys and notes, nobody can censor you. But if the target audience is mass adoption, this is a hard wall. Most users will make a mistake, lose the note, or panic and try to contact support that does not exist.

Designing Private Interfaces

The lesson is not that privacy and good UX are incompatible — it is that building private applications requires re-thinking the interface from scratch rather than bolting ZK onto an existing transparent design. Some strategies that help:

- **Wallet-managed private state.** Instead of the user handling notes manually, the wallet generates and stores them automatically. Midnight's architecture does exactly

this: private state lives encrypted in your local wallet, and the SDK manages proof generation behind the scenes.

- **Progressive disclosure.** Show the user only what they need to see at each step. The complexity of ZK proofs is an implementation detail — it does not belong in the UI.
- **Sensible defaults.** Make the private path the default rather than opt-in. Monero's design (always private) achieves better anonymity sets than Zcash's (opt-in) precisely because users do not have to make a choice.

4.4. E TRADE-OFFS: PERFORMANCE VS. PRIVACY

Introducing ZK privacy fundamentally changes the computational model of your application, and it is worth being explicit about where the costs land.

f-Chain Computation

Ethereum popularised an on-chain computation model: all logic executes on the network, results are public and globally verifiable, and the node is the source of truth. Privacy with ZK inverts this: **all meaningful computation happens off-chain**, and the on-chain component verifies only a proof that the computation was done correctly.

This shift has real consequences:

- **User devices bear the proving cost.** Generating a ZK proof can take seconds to minutes on a mobile device. This is improving rapidly with hardware acceleration and more efficient proof systems, but it is not free.
- **Application logic must be expressed as a circuit.** Not all computations are equally easy to express in arithmetic circuit form. Division, conditionals, and floating-point operations all require careful encoding. Languages like Noir, Circom, and Compact (Midnight) exist to make this easier, but there is still a learning curve.
- **Debugging is harder.** When a proof fails to verify, the error is usually a constraint violation deep inside the circuit — not a friendly stack trace. Tooling is improving but lags behind standard development environments.

usting a Proving Server

One workaround for slow on-device proving is to offload it to a **centralised proof server**. The user sends their inputs to the server, the server generates the proof, and returns it. This is faster, but it introduces a critical trust assumption: you are now telling a third party the very information you are trying to keep private.

> **The rule:** if the proving happens on a server you do not control, and that server can see your private inputs, then you do not have ZK privacy — you have *server-side secrecy*, which is a much weaker guarantee.

The correct architecture is to run the proof server locally. Midnight does this via a local Docker container: your private inputs go to a process on your own machine and never leave it. The tradeoff is that the user must run Docker, which is again a UX challenge.

Summary of Trade-offs

Choice	Benefit	Cost
On-device proving	True privacy	Slow, hardware-dependent
Centralised proof server	Fast, seamless UX	Privacy broken (trust the server)
ZK for compression only	Scalability, no UX friction	Complexity, proving time
Always-on privacy (Monero)	Large anonymity set	No opt-out, regulatory friction
Opt-in privacy (Zcash)	Regulatory flexibility	Small anonymity set

That is enough theory. From the next chapter onwards we will stop talking about ZK and start *building* it: our first ZK smart contract on Cardano using Plutus.